Reproduction et analyse de :
"Pricing Options and Computing Implied Volatilities Using Neural Networks"

Liu, Oosterlee & Bohte (2019) [1]

Dan Allouche

Avril 2026

Déclaration sur l'utilisation d'outils d'IA

Au cours de ce projet, nous avons utilisé **Claude** (Anthropic) comme outil d'assistance. Concrètement, l'IA nous a permis de gagner du temps sur plusieurs aspects :

- *Compréhension d'articles complémentaires* : nous avons utilisé Claude pour synthétiser rapidement le contenu de certains articles de référence (méthode COS de Fang & Oosterlee [3], techniques de LR cyclique de Smith [4]) et accéder aux formules de la fonction caractéristique de Heston.
- *Aide à l'implémentation* : certaines parties du code ont été implémentées avec l'aide de Claude, notamment la parallélisation de la génération Heston via `multiprocessing`, la gestion des différentes planifications de taux d'apprentissage, et les fonctions de visualisation. L'architecture générale du projet (découpage en modules, pipeline d'expériences) a également bénéficié de ses suggestions.
- *Rédaction* : Claude a assisté la mise en forme LaTeX de ce rapport et la structuration du document.

L'ensemble de la **compréhension mathématique** (EDP de Black-Scholes, système de Heston, méthode COS), de la **conception expérimentale** (choix des expériences, paramètres, méthodologie itérative sur le Heston), de l'**interprétation des résultats** et de l'**analyse critique** constitue un travail personnel. En particulier, le diagnostic des problèmes rencontrés — l'inutilité du GPU pour ce modèle, le surapprentissage du Heston, l'impact des outliers — et les décisions prises en conséquence (filtrage, changement de stratégie) relèvent entièrement de notre propre démarche expérimentale.

Table des matières

1 Introduction **3**

2 Contexte mathématique **3**
2.1 Le modèle de Black-Scholes . . . 3
2.2 Volatilité implicite et méthodes de recherche de racines . . . 4
2.3 Le modèle de Heston à volatilité stochastique . . . 4
2.4 La méthode COS . . . 5

3 Méthodologie du réseau de neurones **6**
3.1 Architecture du réseau . . . 6
3.2 Métriques d'évaluation . . . 7

3.3 Taux d'apprentissage 7
3.4 Transformation gradient-squash pour la volatilité implicite 9

4 Résultats numériques **9**
4.1 Configuration 9
4.2 Étude de la taille du dataset (Tableau 3, Figure 6) 9
4.3 BS-ANN (Tableau 5, Figure 7) 10
4.4 IV-ANN (Tableau 7, Figure 8) 11
4.5 Comparaison de vitesse (Tableau 8) 12
4.6 Heston-ANN (Tableau 10, Figure 9) 13
4.7 Surface de volatilité implicite Heston (Tableau 11, Figures 11–12) 14

5 Difficultés rencontrées et choix effectués **16**
5.1 Pourquoi le GPU ne nous a pas aidés 16
5.2 Le défi Heston : cinq tentatives 16
5.3 Récapitulatif des différences 17

6 Propositions d'amélioration **17**

7 Conclusion **18**

1 Introduction

En finance computationnelle, la valorisation des produits dérivés nécessite le calcul rapide de prix d'options pour des milliers de configurations de paramètres. Cette exigence est particulièrement forte dans deux contextes : la *calibration de modèles*, où l'on ajuste les paramètres d'un modèle de prix pour reproduire les prix de marché observés, et la *gestion du risque en temps réel*, où les portefeuilles doivent être revalorisés en continu.

Si le modèle de Black-Scholes admet une solution analytique pour les options européennes, des modèles plus réalistes comme le modèle de Heston à volatilité stochastique requièrent des méthodes numériques coûteuses : résolution d'EDP par différences finies, simulation de Monte Carlo, ou techniques de Fourier comme la méthode COS. Ces méthodes peuvent prendre plusieurs secondes par évaluation, ce qui est prohibitif lorsqu'il faut valoriser des milliers d'options simultanément.

Liu, Oosterlee et Bohte [1] proposent une approche originale : utiliser un réseau de neurones artificiels (ANN) entraîné hors-ligne sur des données générées par un solveur traditionnel, puis utiliser ce réseau entraîné comme un "agent" rapide pour le calcul en ligne. L'ANN, étant une simple séquence de multiplications matricielles, peut traiter des milliers d'options en un seul passage (*batch mode*), offrant des accélérations de l'ordre de 100 à 200 fois par rapport aux méthodes itératives classiques.

Les auteurs testent cette approche sur trois problèmes de complexité croissante :

1. **BS-ANN** : approximation de la formule analytique de Black-Scholes (4 paramètres d'entrée),
2. **IV-ANN** : calcul de la volatilité implicite de Black-Scholes avec une transformation de gradient (4 entrées),
3. **Heston-ANN** : approximation des prix d'options Heston calculés par la méthode COS (8 entrées).

Dans ce rapport, nous reproduisons ces résultats en Python et PyTorch sur un Apple MacBook Air M3. Nous discutons en détail les difficultés techniques rencontrées — notamment l'impossibilité d'exploiter le GPU pour ce type de modèle — et proposons des améliorations de la méthodologie.

2 Contexte mathématique

2.1 Le modèle de Black-Scholes

Le modèle de Black-Scholes repose sur l'hypothèse que le prix de l'actif sous-jacent suit un mouvement brownien géométrique :

$$dS_t = \mu S_t \, dt + \sigma S_t \, dW_t^s, \tag{1}$$

où μ est le drift, σ la volatilité (supposée constante), et W^s un processus de Wiener standard. Par valorisation risque-neutre et application du lemme d'Itô, le prix d'une option européenne $V(t, S)$ satisfait l'équation aux dérivées partielles de Black-Scholes :

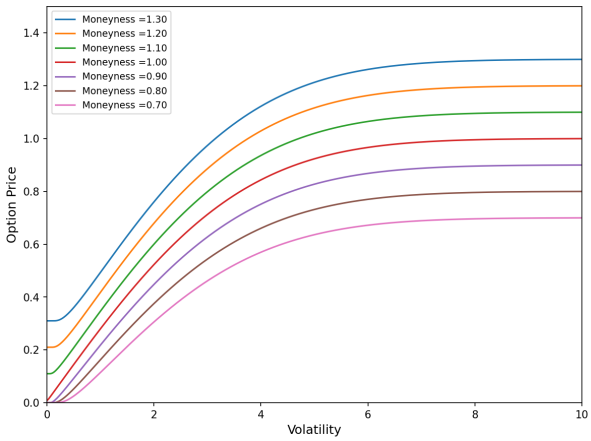
$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} - rV = 0, \tag{2}$$

avec la condition terminale $V(T, S) = \max(S - K, 0)$ pour un call européen. Cette EDP admet la solution analytique célèbre :

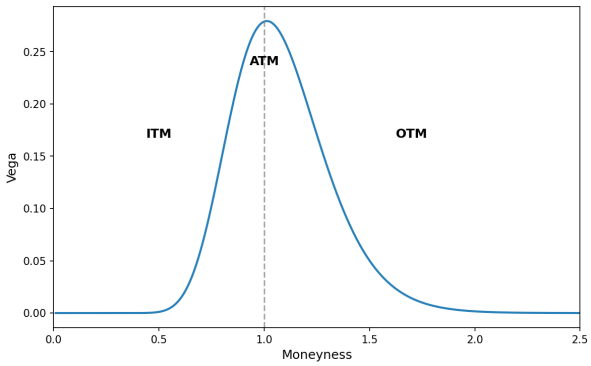
$$V_c = SN(d_1) - Ke^{-r\tau}N(d_2), \quad d_1 = \frac{\ln(S/K) + (r + \frac{1}{2}\sigma^2)\tau}{\sigma\sqrt{\tau}}, \quad d_2 = d_1 - \sigma\sqrt{\tau}, \tag{3}$$

où $\tau = T - t$ est le temps restant jusqu'à maturité et $N(\cdot)$ est la fonction de répartition de la loi normale standard. La Figure 1 illustre le comportement de cette formule : le prix de l'option croît avec la volatilité (Figure 1a), ce qui est intuitif puisqu'une plus grande incertitude augmente la valeur de l'optionalité. La sensibilité du prix à la volatilité, appelée *Vega*, varie considérablement

selon le moneyness de l'option (Figure 1b) : elle est maximale pour les options à la monnaie (ATM) et tend vers zéro pour les options très dans la monnaie (ITM) ou hors de la monnaie (OTM).



(a) Le prix croît avec σ pour tout moneyness.



(b) Le Vega est maximal en ATM et s'annule en deep ITM/OTM.

FIGURE 1 – Prix et Vega en fonction de la volatilité et du moneyness (Figure 1 de [1]).

2.2 Volatilité implicite et méthodes de recherche de racines

En pratique, la volatilité σ n'est pas directement observable sur les marchés. On observe les prix des options, et on en déduit la *volatilité implicite* σ^* en inversant la formule de Black-Scholes : trouver σ^* tel que $BS(\sigma^*; S, K, \tau, r) = V^{mkt}$. Il n'existe pas d'inverse analytique, ce problème se ramène donc à une recherche de racine : $g(\sigma) = BS(\sigma) - V^{mkt} = 0$.

Newton-Raphson (Eq. 9 de l'article) : cette méthode utilise la dérivée analytique (le Vega) pour converger quadratiquement :

$$\sigma_{k+1}^* = \sigma_k^* - \frac{V(\sigma_k^*) - V^{mkt}}{\text{Vega}(\sigma_k^*)}, \qquad \text{Vega} = S\sqrt{\tau}\,\phi(d_1) \tag{4}$$

où $\phi(\cdot)$ désigne la densité de la loi normale standard (ne pas confondre avec la fonction caractéristique φ utilisée dans la méthode COS). La convergence de Newton-Raphson est très rapide (quelques itérations) mais elle échoue lorsque le Vega est proche de zéro (deep ITM/OTM), car la division par un nombre très petit provoque une divergence.

Méthode de la sécante (Eq. 11) : elle remplace la dérivée analytique par une approximation par différences finies :

$$\sigma_{k+1} = \sigma_k - g(\sigma_k) \cdot \frac{\sigma_k - \sigma_{k-1}}{g(\sigma_k) - g(\sigma_{k-1})}. \tag{5}$$

L'avantage est de ne pas nécessiter le calcul du Vega, mais elle souffre des mêmes problèmes de stabilité que Newton-Raphson lorsque deux itérés consécutifs sont proches.

Bisection : elle divise systématiquement l'intervalle de recherche en deux à chaque itération. Elle est lente ($\sim$50 itérations) mais garantit la convergence dès lors qu'on dispose d'un encadrement initial $[a, b]$ avec $g(a) \cdot g(b) < 0$.

Méthode de Brent (Eq. 10) : c'est la méthode recommandée dans l'article. Elle combine intelligemment la bisection (pour la robustesse), l'interpolation quadratique inverse (pour la vitesse) et la méthode de la sécante. Elle converge toujours si un encadrement existe, tout en étant presque aussi rapide que Newton-Raphson.

2.3 Le modèle de Heston à volatilité stochastique

L'hypothèse de volatilité constante dans Black-Scholes est contredite par les marchés : les volatilités implicites observées dépendent du strike et de la maturité, formant ce qu'on appelle le *smile*

ou *skew* de volatilité. Pour capturer ce phénomène, Heston [2] a proposé en 1993 un modèle où la variance ν_t est elle-même un processus stochastique :

$$dS_t = rS_t\,dt + \sqrt{\nu_t}\,S_t\,dW_t^s, \quad S_{t_0} = S_0, \tag{6}$$

$$d\nu_t = \kappa(\bar{\nu} - \nu_t)\,dt + \gamma\sqrt{\nu_t}\,dW_t^\nu, \quad \nu_{t_0} = \nu_0, \tag{7}$$

$$dW_t^s\,dW_t^\nu = \rho\,dt. \tag{8}$$

Le processus de variance est un processus de Cox-Ingersoll-Ross à retour à la moyenne : κ contrôle la vitesse à laquelle ν_t revient vers sa valeur long terme $\bar{\nu}$, γ est la volatilité de la variance ("vol of vol"), et ρ est la corrélation entre les aléas du prix et de la variance. En pratique, $\rho < 0$ (effet de levier : quand le prix baisse, la volatilité augmente), ce qui génère le skew observé sur les marchés.

Ce modèle à deux facteurs mène à l'EDP de pricing bidimensionnelle suivante :

$$\frac{\partial V}{\partial t} + rS\frac{\partial V}{\partial S} + \kappa(\bar{\nu} - \nu)\frac{\partial V}{\partial \nu} + \frac{1}{2}\nu S^2\frac{\partial^2 V}{\partial S^2} + \rho\gamma S\nu\frac{\partial^2 V}{\partial S\partial \nu} + \frac{1}{2}\gamma^2\nu\frac{\partial^2 V}{\partial \nu^2} - rV = 0, \tag{9}$$

qui n'admet pas de solution analytique directe. Cependant, la fonction caractéristique du processus log-prix est connue en forme fermée, ce qui ouvre la porte aux méthodes de Fourier.

2.4 La méthode COS

La méthode COS, introduite par Fang et Oosterlee [3], est une technique de pricing par transformée de Fourier particulièrement efficace. L'idée est d'approcher la densité risque-neutre $f(x)$ du log-prix $x = \ln(S_T/K)$ par une série de Fourier-cosinus sur un intervalle tronqué $[a, b]$:

$$f(x) \approx \sum_{k=0}^{N-1}{}' A_k \cos\left(k\pi\frac{x-a}{b-a}\right), \tag{10}$$

où les coefficients A_k sont reliés à la fonction caractéristique :

$$A_k = \frac{2}{b-a}\,\mathrm{Re}\left[\varphi\left(\frac{k\pi}{b-a}\right)e^{-ik\pi\frac{a}{b-a}}\right]. \tag{11}$$

Pour le modèle de Heston, la fonction caractéristique $\varphi(u) = E[e^{iu\cdot x}]$ est connue analytiquement :

$$\varphi(u) = \exp(C(u) + D(u)\,\nu_0), \tag{12}$$

avec les expressions suivantes :

$$d = \sqrt{(\rho\gamma iu - \kappa)^2 + \gamma^2(iu + u^2)}, \tag{13}$$

$$g = \frac{\kappa - \rho\gamma iu - d}{\kappa - \rho\gamma iu + d}, \tag{14}$$

$$C(u) = iu\,r\tau + \frac{\kappa\bar{\nu}}{\gamma^2}\left[(\kappa - \rho\gamma iu - d)\tau - 2\ln\frac{1 - ge^{-d\tau}}{1-g}\right], \tag{15}$$

$$D(u) = \frac{\kappa - \rho\gamma iu - d}{\gamma^2}\cdot\frac{1 - e^{-d\tau}}{1 - ge^{-d\tau}}. \tag{16}$$

Le prix d'un call européen est alors :

$$V = Ke^{-r\tau}\sum_{k=0}^{N-1}{}'\mathrm{Re}\left[\varphi\left(\frac{k\pi}{b-a}\right)e^{i\frac{k\pi}{b-a}x_0}\cdot e^{-ik\pi\frac{a}{b-a}}\right]V_k, \qquad x_0 = \ln(S_0/K), \tag{17}$$

où $V_k = \frac{2}{b-a}(\chi_k - \psi_k)$ sont les coefficients analytiques du payoff du call sur l'intervalle $[0, b]$, et le symbole $'$ indique que le premier terme ($k = 0$) est divisé par 2. Les coefficients χ_k et ψ_k s'expriment analytiquement :

$$\chi_k(c,d) = \frac{1}{1+\omega_k^2}\left[\cos(\omega_k(d-a))e^d - \cos(\omega_k(c-a))e^c + \omega_k \sin(\omega_k(d-a))e^d - \omega_k \sin(\omega_k(c-a))e^c\right], \tag{18}$$

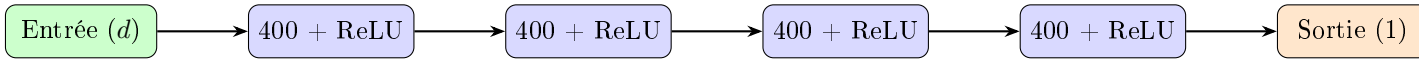
$$\psi_k(c,d) = \begin{cases} \frac{1}{\omega_k}\left[\sin(\omega_k(d-a)) - \sin(\omega_k(c-a))\right] & \text{si } k > 0, \\ d - c & \text{si } k = 0, \end{cases} \tag{19}$$

avec $\omega_k = k\pi/(b-a)$. L'intervalle de troncature est déterminé par les cumulants : $a = c_1 - L\sqrt{c_2}$, $b = c_1 + L\sqrt{c_2}$, avec $L = 50$. Nous utilisons $N = 1500$ termes de Fourier. Pour les calls deep OTM, dont le prix est très proche de zéro et sujet à des erreurs de troncature, nous calculons d'abord le prix du put (plus grand et plus stable numériquement) puis appliquons la parité call-put : $C = P + S - Ke^{-r\tau}$.

3 Méthodologie du réseau de neurones

3.1 Architecture du réseau

Le réseau utilisé est un perceptron multicouche (MLP) dont l'architecture, sélectionnée par les auteurs via une recherche aléatoire d'hyperparamètres avec validation croisée 3-fold, est décrite dans le Tableau 2 de l'article. Le réseau comporte **4 couches cachées de 400 neurones** chacune, soit la structure suivante :



où $d = 4$ pour BS-ANN et IV-ANN, et $d = 8$ pour Heston-ANN. Chaque couche calcule $\mathbf{z}^{(l)} = \mathrm{ReLU}(\mathbf{W}^{(l)} \cdot \mathbf{z}^{(l-1)} + \mathbf{b}^{(l)})$, sauf la dernière couche qui n'a pas d'activation (régression).

Fonction d'activation ReLU. $\mathrm{ReLU}(x) = \max(0, x)$. Contrairement à la sigmoïde ou la tangente hyperbolique, ReLU ne souffre pas du problème de disparition du gradient pour les valeurs positives, ce qui accélère la convergence. Son inconvénient (les neurones "morts" pour $x < 0$) est mitigé par l'initialisation adéquate.

Initialisation Xavier (Glorot uniform). Les poids sont tirés de $\mathcal{U}(-a, a)$ avec $a = \sqrt{6/(n_{in} + n_{out})}$. Cette initialisation maintient la variance des activations constante à travers les couches, évitant l'explosion ou la disparition des signaux en début d'entraînement. Les biais sont initialisés à zéro.

Optimiseur Adam. Adam combine deux idées : le *momentum* (moyenne mobile des gradients passés pour lisser la trajectoire d'optimisation) et *RMSprop* (normalisation par la moyenne mobile des gradients carrés pour adapter le pas à chaque paramètre). La règle de mise à jour est :

$$W \leftarrow W - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}, \qquad \hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \tag{20}$$

où m_t et v_t sont les moyennes mobiles des gradients et des gradients carrés. Adam est particulièrement adapté aux problèmes avec beaucoup de paramètres et des gradients bruités.

Taille de batch. Un batch de 1024 échantillons est utilisé. Ce choix représente un compromis : un batch trop petit rend le gradient trop bruité, un batch trop grand réduit l'effet de régularisation implicite du bruit stochastique et ralentit la convergence en nombre d'époques.

Absence de dropout et batch normalization. L'article constate que ni le dropout ni la batch normalization n'améliorent les résultats pour ce problème de régression. Le dropout, qui désactive aléatoirement une fraction des neurones pendant l'entraînement, est efficace pour les problèmes de classification avec des features éparses (images), mais peut nuire à la régression de

fonctions lisses. La batch normalization, qui normalise les activations par la moyenne et la variance du batch, n'est pas adaptée ici car la valeur de sortie est très sensible aux variations des paramètres d'entrée — contrairement aux features éparses d'une image, où la normalisation par lot est efficace.

3.2 Métriques d'évaluation

Les modèles sont évalués à l'aide de cinq métriques complémentaires (Eq. 20–21 de l'article) :

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \qquad \mathrm{RMSE} = \sqrt{\mathrm{MSE}}, \tag{21}$$

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|, \qquad \mathrm{MAPE} = \frac{1}{n}\sum_{i=1}^{n}\frac{|y_i - \hat{y}_i|}{|y_i|}, \tag{22}$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}. \tag{23}$$

Le MSE est utilisé comme fonction de perte pendant l'entraînement. Le R^2 mesure la fraction de la variance expliquée par le modèle : $R^2 = 1$ correspond à une prédiction parfaite, $R^2 = 0$ à un modèle qui prédit la moyenne.

3.3 Taux d'apprentissage

Le taux d'apprentissage η est le paramètre le plus critique de l'entraînement. L'article utilise la méthode de Smith [4] pour déterminer la plage optimale : on commence avec un η très petit (10^{-9}) et on l'augmente exponentiellement à chaque batch, en enregistrant la perte. La Figure 2 montre le résultat : la perte décroît fortement entre 10^{-5} et 10^{-3}, stagne pour des valeurs plus petites, et remonte au-delà de 10^{-3}, indiquant une instabilité de l'optimisation.

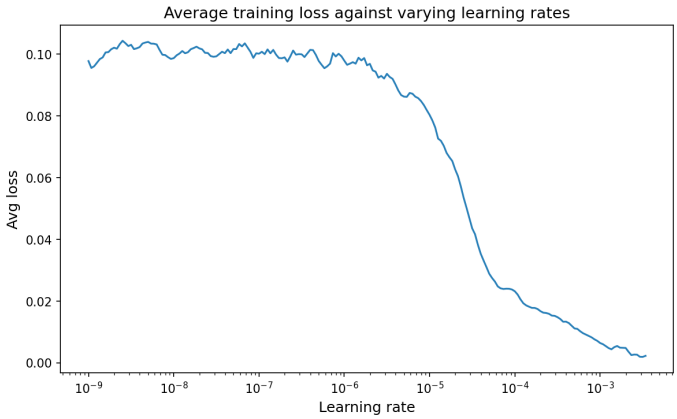


FIGURE 2 – Test de plage de LR (Figure 3 de [1]). La zone de pente maximale entre 10^{-5} et 10^{-3} définit la plage optimale.

L'article utilise ensuite une décroissance exponentielle : $\eta_t = \eta_0 \cdot \gamma^t$ avec $\gamma = (10^{-5}/10^{-3})^{1/3000}$, partant de 10^{-3} et arrivant à 10^{-5} après 3000 époques. L'idée est de commencer avec un grand pas pour explorer rapidement l'espace des paramètres, puis de réduire pour affiner la convergence.

Nous avons comparé trois planifications sur 1M d'échantillons Heston filtrés (Figure 3) : constant ($\eta = 10^{-3}$), décroissant, et cyclique (oscillant entre 10^{-5} et 10^{-3}). Les résultats (Tableau 1) montrent que le cyclique atteint à la fois le meilleur MSE d'entraînement ($1{,}82 \times 10^{-6}$) et le meilleur MSE de validation ($1{,}65 \times 10^{-4}$). En termes de MSE de validation, le cyclique est le meilleur ($1{,}65 \times 10^{-4}$), suivi du constant ($1{,}69 \times 10^{-4}$), puis du décroissant ($1{,}78 \times 10^{-4}$). Le décroissant, malgré une descente initiale rapide (visible sur la Figure 3), est le moins bon des trois en validation ($1{,}78 \times 10^{-4}$), bien qu'il surpasse le constant en entraînement ($4{,}04 \times 10^{-6}$ vs

$5{,}53 \times 10^{-6}$). La Figure 4 montre les courbes de perte détaillées : on observe que le LR cyclique produit des oscillations caractéristiques qui aident potentiellement à sortir des minima locaux.

TABLE 1 – MSE final pour trois planifications de LR (1M échantillons Heston filtrés, 3000 époques).

Planification	**MSE train**	**MSE val**
Constant ($\eta = 10^{-3}$)	$5{,}53 \times 10^{-6}$	$1{,}69 \times 10^{-4}$
Décroissant ($10^{-3} \to 10^{-5}$)	$4{,}04 \times 10^{-6}$	$1{,}78 \times 10^{-4}$
Cyclique	$1{,}82 \times 10^{-6}$	$1{,}65 \times 10^{-4}$

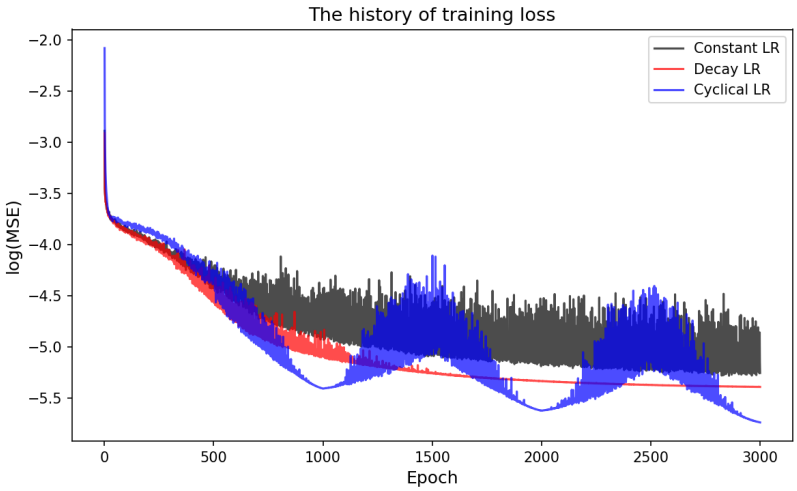


FIGURE 3 – Comparaison des trois planifications de LR sur le modèle Heston (Figure 4). Le décroissant descend rapidement en début d'entraînement, mais le cyclique atteint le meilleur MSE final.

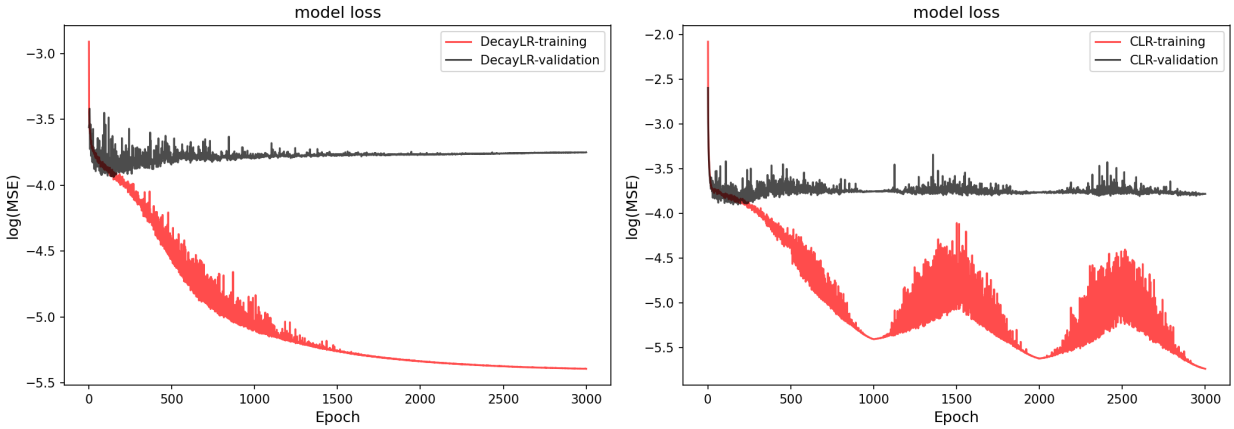


FIGURE 4 – Détail des pertes d'entraînement (rouge) et validation (noir) pour le LR décroissant (gauche) et cyclique (droite). Le cyclique atteint un MSE de validation légèrement inférieur, bien que l'écart train/val soit similaire à celui du constant.

Cette comparaison utilise 1M d'échantillons Heston filtrés, conformément à l'article. Les trois entraînements séquentiels ont nécessité environ 21 heures sur CPU. Bien que le LR cyclique donne les meilleurs résultats dans cette comparaison, nous avons conservé la planification décroissante pour nos modèles finaux, conformément au choix de l'article, afin de garantir la comparabilité

directe des résultats. Avec le recul, un ré-entraînement du Heston-ANN avec le LR cyclique serait une amélioration naturelle à explorer.

3.4 Transformation gradient-squash pour la volatilité implicite

Le mapping direct prix $\rightarrow$ volatilité pose un problème fondamental : la relation entre V/K et σ^* présente des gradients très raides dans certaines régions (options deep ITM), rendant l'apprentissage difficile pour le réseau. L'article propose une transformation en trois étapes :

1. **Soustraction de la valeur intrinsèque** : $\hat{V} = V - \max(S - Ke^{-r\tau}, 0)$. On isole la *valeur temps* de l'option, éliminant la composante déterministe du prix pour les options ITM.
2. **Transformation logarithmique** : on utilise $\log(\hat{V}/K)$ comme entrée au lieu de V/K. Le logarithme compresse les grandes valeurs et étale les petites, réduisant considérablement la dynamique du signal.
3. **Filtrage** : les échantillons avec $\hat{V} < 10^{-7}$ sont éliminés pour éviter les valeurs $\log \rightarrow -\infty$.

Remarque : ce filtrage ($\hat{V} < 10^{-7}$) est spécifique à l'IV-ANN et porte sur la *valeur temps* $\hat{V}$. Il ne doit pas être confondu avec le filtrage appliqué au Heston-ANN (Section 4.6), qui porte sur le *prix* $V \in (10^{-6}, 0{,}67)$ conformément au Tableau 9 de l'article.

Cette transformation améliore le MSE d'un facteur **116** dans nos expériences. Notons que ce facteur reflète à la fois l'effet de la transformation *et* le nombre d'époques supplémentaires (3000 vs 1000). Par comparaison, les chiffres de l'article (MSE de $6{,}36 \times 10^{-4}$ sans transformation à $1{,}55 \times 10^{-8}$ avec) indiquent un gain beaucoup plus important. La différence s'explique par le fait que notre modèle "sans transformation" (MSE $= 1{,}16 \times 10^{-5}$) est déjà significativement meilleur que celui de l'article (MSE $= 6{,}36 \times 10^{-4}$), pour des raisons qui restent incertaines (différences de framework, de nombre d'époques pour la baseline, ou de graine aléatoire).

4 Résultats numériques

4.1 Configuration

Toutes les expériences ont été exécutées sur un **Apple MacBook Air M3** (8 cœurs CPU, 16 Go de mémoire unifiée) avec Python 3.14 et PyTorch 2.11.0. L'entraînement a été réalisé **uniquement sur CPU** (voir Section 5.1 pour la justification détaillée de ce choix). La génération des données Heston a été parallélisée sur les 8 cœurs via `multiprocessing.Pool`.

Conformément à l'article, nous utilisons **1 000 000 d'échantillons LHS** pour tous les modèles, **3000 époques** d'entraînement avec un batch de 1024, et la décroissance exponentielle du taux d'apprentissage de 10^{-3} à 10^{-5}. Pour BS-ANN et IV-ANN, nous appliquons le re-entraînement sur l'ensemble complet (train+val) mentionné page 9 de l'article, sous la forme d'un *fine-tuning* de 200 époques supplémentaires à taux d'apprentissage constant $\eta = 10^{-5}$. Le re-entraînement sur train+val a également été appliqué au Heston-ANN lors de notre dernière itération (Section 4.6).

4.2 Étude de la taille du dataset (Tableau 3, Figure 6)

Avant de procéder aux expériences principales, nous avons reproduit l'étude de l'impact de la taille du dataset (Section 4.1 de l'article). Le principe est le suivant : on fixe le problème (IV-ANN avec gradient-squash), l'architecture et les hyperparamètres, puis on fait varier uniquement la taille de l'ensemble d'entraînement. Le cas de référence (Case 3) utilise 24 300 échantillons ; les autres cas multiplient cette taille par $\frac{1}{8}$, $\frac{1}{4}$, $\frac{1}{2}$, 1, 2, 4 et 8, allant de 3 037 à 194 400 échantillons. Pour chaque taille, on entraîne 5 modèles avec des graines aléatoires différentes (200 époques chacun) et on évalue sur un ensemble de test *fixe*, ce qui permet de mesurer à la fois la performance moyenne et sa variance.

La Figure 5 montre deux tendances nettes : d'une part, le R^2 (axe droit) augmente de manière monotone, passant de 94,5 % avec 3 037 échantillons à 99,998 % avec 194 400. D'autre part, les barres d'erreur (représentant la variance sur 5 seeds) diminuent avec la taille, indiquant que les résultats

deviennent plus reproductibles. On observe également que l'écart entre le MSE d'entraînement (bleu) et de test (rouge) se réduit, signe que le surapprentissage diminue avec plus de données. Cette étude a guidé notre décision d'utiliser 1M échantillons pour les modèles finaux.

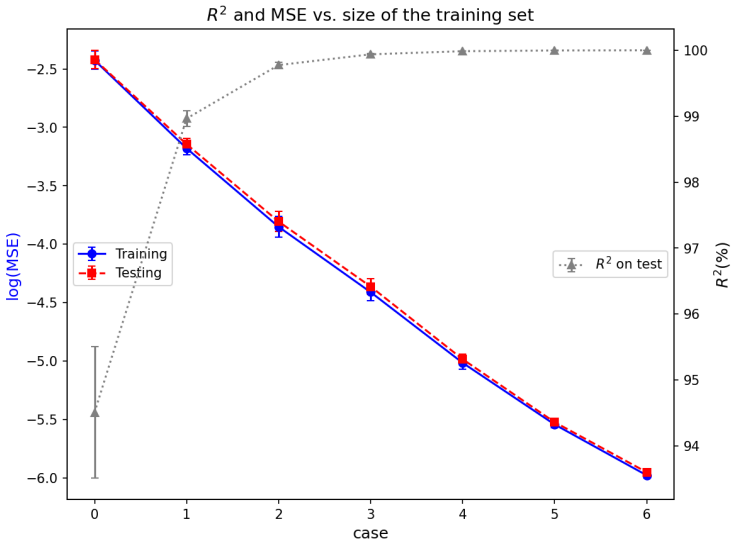


FIGURE 5 – Le R^2 (axe droit) augmente et le MSE (axe gauche) diminue avec la taille du dataset. Les barres d'erreur (5 seeds) se résorbent avec plus de données (Figure 6).

4.3 BS-ANN (Tableau 5, Figure 7)

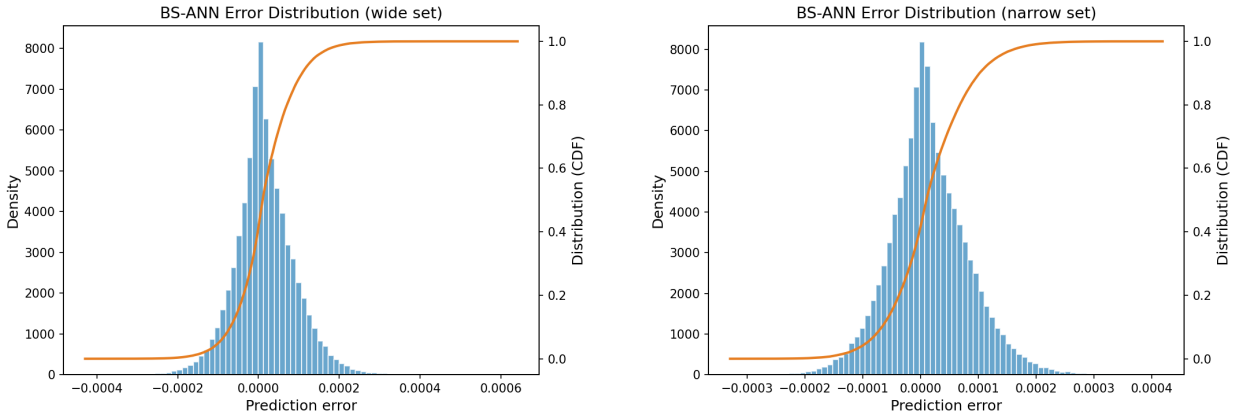
Le premier modèle apprend directement la formule analytique de Black-Scholes. L'entrée du réseau est le quadruplet $\{S_0/K, \tau, r, \sigma\}$ et la sortie est le prix normalisé V/K. La normalisation par le strike K permet de réduire la dimension du problème : au lieu de 5 variables $(S_0, K, \tau, r, \sigma)$, on travaille avec 4 variables en exploitant l'homogénéité de la formule de BS. Les plages de paramètres (Tableau 4 de l'article) couvrent un large spectre de conditions de marché : moneyness de 0,4 à 1,6, maturité de 0,2 à 1,1 an, taux de 2 % à 10 %, et volatilité de 1 % à 100 %.

L'évaluation est faite sur deux ensembles de test : un ensemble *wide* (mêmes plages que l'entraînement) et un ensemble *narrow* (plages légèrement restreintes). L'idée est que le réseau est généralement plus précis au centre du domaine qu'aux bords, et l'ensemble narrow permet de quantifier cet effet.

TABLE 2 – Performance du BS-ANN (1M échantillons, reproduction du Tableau 5). Les R^2 sont arrondis à 7 décimales ; ils diffèrent légèrement au-delà.

	MSE	**RMSE**	**MAE**	**R^2**
Notre : Train (wide)	$5{,}32 \times 10^{-9}$	$7{,}30 \times 10^{-5}$	$5{,}43 \times 10^{-5}$	0,9999999
Notre : Test (wide)	$5{,}54 \times 10^{-9}$	$7{,}44 \times 10^{-5}$	$5{,}53 \times 10^{-5}$	0,9999999
Notre : Test (narrow)	$4{,}92 \times 10^{-9}$	$7{,}02 \times 10^{-5}$	$5{,}29 \times 10^{-5}$	0,9999999
Article : Test (wide)	$8{,}21 \times 10^{-9}$	$9{,}06 \times 10^{-5}$	$6{,}79 \times 10^{-5}$	—
Article : Test (narrow)	$7{,}00 \times 10^{-9}$	$8{,}37 \times 10^{-5}$	$6{,}49 \times 10^{-5}$	—

Nos résultats sont légèrement meilleurs que ceux de l'article (MSE $5{,}54 \times 10^{-9}$ contre $8{,}21 \times 10^{-9}$). Cette différence est probablement due au bruit statistique (graines aléatoires différentes, framework PyTorch vs TensorFlow). La Figure 6 montre que la distribution des erreurs est approximativement normale et centrée sur zéro, avec des erreurs maximales inférieures à 5×10^{-4}. L'ensemble de test narrow (plage de paramètres plus restreinte) donne de meilleurs résultats, confirmant l'effet de bord aux frontières du domaine d'entraînement.



(a) Test wide : erreurs jusqu'à $\pm 5 \times 10^{-4}$.

(b) Test narrow : distribution plus resserrée.

FIGURE 6 – Distributions d'erreur du BS-ANN (Figure 7). La courbe orange est la CDF cumulée.

4.4 IV-ANN (Tableau 7, Figure 8)

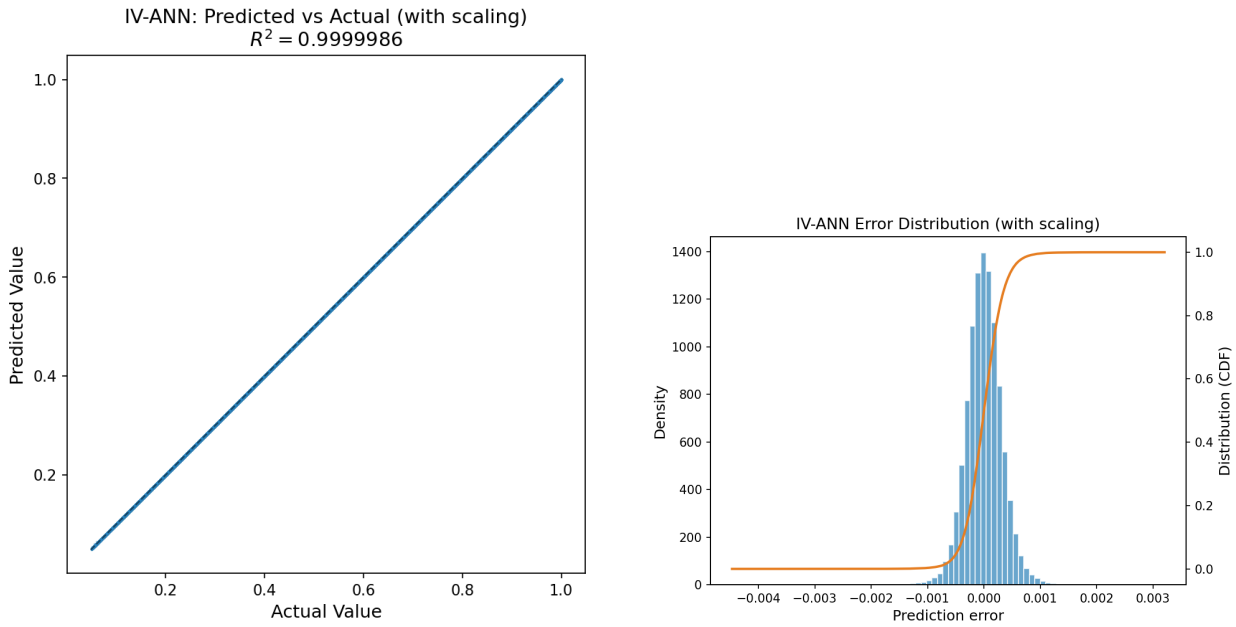
L'objectif est d'apprendre la fonction inverse de Black-Scholes : étant donné un prix d'option, retrouver la volatilité qui l'a généré. Pour construire le dataset, nous procédons en sens inverse : nous générons aléatoirement des quadruplets $(S_0/K, \tau, r, \sigma)$ par LHS, nous calculons le prix V correspondant par la formule de Black-Scholes, puis nous prenons le σ utilisé comme « volatilité réelle » (vérité terrain). Le réseau reçoit en entrée les variables de marché et le prix (transformé par gradient-squash), et doit prédire le σ d'origine. C'est donc un problème d'inversion de fonction : le réseau apprend BS^{-1}.

Nous avons entraîné deux modèles pour comparer l'effet de la transformation gradient-squash : le premier reçoit directement V/K en entrée (1000 époques), le second reçoit $\log(\hat{V}/K)$ après soustraction de la valeur intrinsèque (3000 époques).

TABLE 3 – Performance du IV-ANN avec et sans gradient-squash (1M échantillons).

Entrée	**MSE**	**MAE**	**MAPE**	$\mathbf{R^2}$
$m, \tau, r, V/K \to \sigma^*$	$1{,}16 \times 10^{-5}$	$1{,}43 \times 10^{-3}$	$5{,}92 \times 10^{-3}$	0,99983
$m, \tau, r, \log(\hat{V}/K) \to \sigma^*$	$\mathbf{9{,}96 \times 10^{-8}}$	$\mathbf{2{,}41 \times 10^{-4}}$	$\mathbf{6{,}47 \times 10^{-4}}$	**0,9999986**
Article (sans)	$6{,}36 \times 10^{-4}$	$1{,}24 \times 10^{-2}$	$7{,}67 \times 10^{-2}$	0,97510
Article (avec)	$1{,}55 \times 10^{-8}$	$9{,}73 \times 10^{-5}$	$2{,}11 \times 10^{-3}$	0,9999998

La Figure 7a montre un alignement quasi parfait entre les volatilités prédites et réelles ($R^2 = 0{,}9999986$). La distribution des erreurs (Figure 7b) est centrée sur zéro avec une déviation maximale inférieure à 5×10^{-3}, confirmant que le réseau a bien appris la relation inverse.



(a) Prédit vs réel : quasi-parfait.

(b) Erreurs centrées sur zéro.

FIGURE 7 – Performance du IV-ANN avec gradient-squash (Figure 8).

4.5 Comparaison de vitesse (Tableau 8)

Nous avons comparé cinq méthodes sur 20 000 options avec un moneyness variant de 0,5 à 1,5, incluant des options deep ITM et deep OTM. Les options dont le prix est inférieur à 10^{-8} sont préalablement éliminées car elles correspondent à des cas où la précision machine ne permet plus de distinguer le prix de zéro. L'intérêt de cette expérience est double : mesurer le gain de vitesse de l'ANN par rapport aux méthodes itératives, et vérifier sa robustesse dans des conditions difficiles. Les méthodes itératives traitent chaque option séquentiellement (une boucle Python), tandis que l'ANN traite les 20 000 en un seul *forward pass* batché — une unique multiplication matricielle.

TABLE 4 – Comparaison de vitesse sur 20 000 options (moneyness 0,5–1,5).

Méthode	**Temps CPU (sec)**	**Robuste**
Newton-Raphson	6,24	Non
Brent	7,85	Non
Sécante	5,41	Non
Bisection	20,44	Non
IV-ANN	**0,03**	**Oui**

L'IV-ANN est **208 fois plus rapide** que Newton-Raphson (et 180 fois plus rapide que la Sécante, la méthode itérative la plus rapide). Fait notable, cette accélération sur notre CPU Apple M3 dépasse même le facteur ∼100× rapporté par l'article sur GPU (NVIDIA P100), ce qui s'explique probablement par les progrès de l'architecture CPU (mémoire unifiée du M3) et du framework PyTorch depuis 2019. Toutes les méthodes itératives échouent sur certaines options de notre plage de test (moneyness 0,5–1,5) : Newton-Raphson et la Sécante divergent quand le Vega est proche de zéro. Brent et la Bisection, bien que théoriquement robustes dès lors qu'un encadrement existe, échouent également sur certaines options restantes après le pré-filtrage : le seuil $V > 10^{-8}$ élimine les prix strictement nuls, mais des options avec un prix légèrement supérieur (de l'ordre de 10^{-7}) passent le filtre tout en restant trop petites pour que l'encadrement fixe $[\sigma_a, \sigma_b] = [0{,}001; 5{,}0]$ contienne la racine — c'est un problème de *données* plutôt que de l'algorithme lui-même. Notons

que la plage de moneyness utilisée ici (0,5–1,5), bien que légèrement plus étroite que celle du BS-ANN (0,4–1,6), inclut suffisamment d'options deep ITM et deep OTM pour que ces problèmes surviennent. L'IV-ANN, en revanche, produit toujours un résultat en un seul passage batch.

4.6 Heston-ANN (Tableau 10, Figure 9)

Le Heston-ANN est le modèle le plus ambitieux : il doit apprendre une fonction à 8 variables ($m = S_0/K$, τ, r, ρ, κ, $\bar{\nu}$, γ, ν_0) vers le prix de l'option V, avec un strike fixé $K = 1$. La difficulté par rapport au BS-ANN (4 variables) est que l'espace des paramètres est considérablement plus vaste : chaque dimension supplémentaire multiplie le volume à couvrir, nécessitant davantage de données pour éviter le surapprentissage (phénomène connu sous le nom de *fléau de la dimension*).

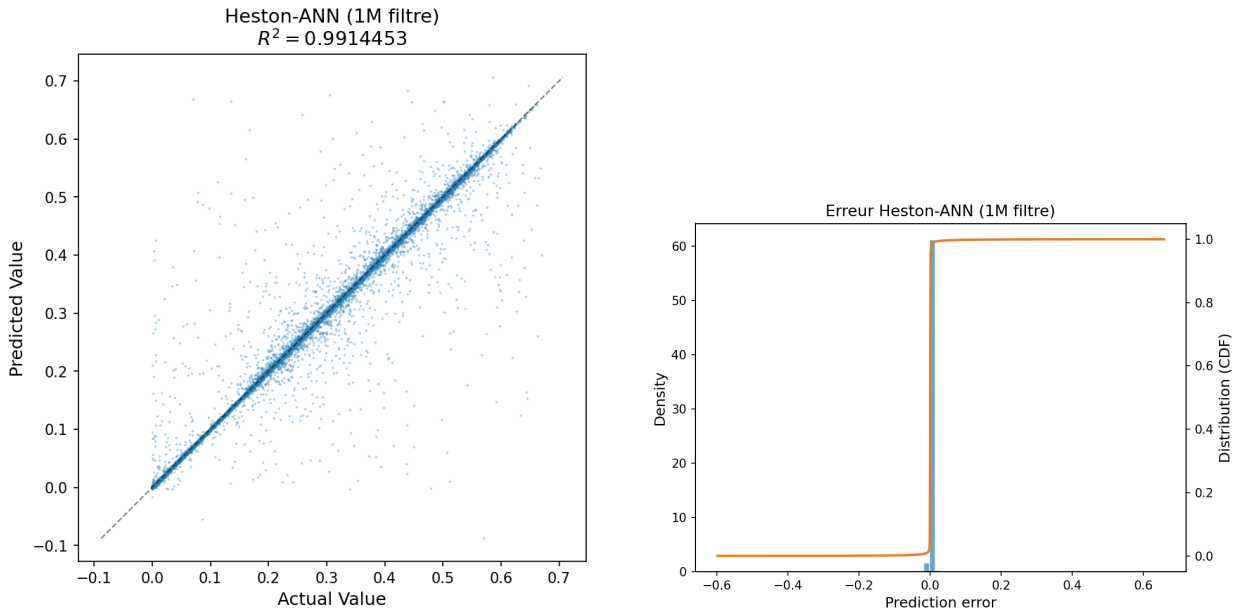
Nous avons généré 1 000 000 d'échantillons par la méthode COS ($N = 1500$, $L = 50$), parallélisée sur 8 cœurs (∼3 minutes). Les plages de paramètres, définies dans le Tableau 9 de l'article, sont : $m \in (0{,}6; 1{,}4)$, $\tau \in (0{,}1; 1{,}4)$, $r \in (0; 10\,\%)$, $\rho \in (-0{,}95; 0)$, $\kappa \in [0{,}01; 2)$, $\bar{\nu} \in [0{,}01; 0{,}5)$, $\gamma \in [0{,}01; 0{,}5)$, $\nu_0 \in (0{,}05; 0{,}5)$. Ces plages couvrent un large spectre de conditions de marché : des smiles très prononcés (ρ très négatif) aux surfaces plates ($\rho \approx 0$), et des marchés calmes ($\bar{\nu}$ petit) aux marchés très volatils ($\bar{\nu}$ grand).

Le Tableau 9 spécifie également une plage de sortie $(0, 0{,}67)$; nous avons donc filtré les prix à $V \in (10^{-6}, 0{,}67)$, obtenant 981 068 échantillons valides sur les 1 000 000 générés (soit ∼1,9 % d'échantillons éliminés, incluant à la fois des prix très proches de zéro pour les options deep OTM et des prix supérieurs à 0,67 pour certaines combinaisons de paramètres extrêmes).

Table 5 – Performance du Heston-ANN (1M filtré, reproduction du Tableau 10).

	MSE	**MAE**	**MAPE**	$\mathbf{R^2}$
Notre : Train	$4{,}26 \times 10^{-6}$	$7{,}12 \times 10^{-4}$	$2{,}19 \times 10^{-1}$	0,9998
Notre : Test	$2{,}01 \times 10^{-4}$	$1{,}76 \times 10^{-3}$	$4{,}47 \times 10^{-1}$	0,9914
Article : Train	$1{,}34 \times 10^{-8}$	$8{,}92 \times 10^{-5}$	$5{,}66 \times 10^{-4}$	0,9999994
Article : Test	$1{,}65 \times 10^{-8}$	$9{,}51 \times 10^{-5}$	$6{,}27 \times 10^{-4}$	0,9999993

La Figure 8a montre un nuage de points globalement aligné sur la diagonale ($R^2 = 0{,}9914$), mais avec une dispersion significative par rapport à l'article. La Figure 8b révèle des erreurs allant jusqu'à ±0,4, bien plus larges que pour BS ou IV. L'impact sur le MAPE est particulièrement marqué : pour les options dont le prix est proche de notre borne basse ($V \approx 10^{-6}$), même une erreur absolue modérée produit une erreur relative très élevée, ce qui explique l'écart d'un facteur ∼700 avec l'article sur cette métrique. La courbe de perte (Figure 9) montre clairement le surapprentissage : la perte de validation stagne après ∼300 époques tandis que la perte d'entraînement continue de décroître. L'analyse détaillée de cet écart et de nos tentatives d'amélioration est présentée en Section 5.2.



(a) Dispersion visible mais tendance correcte.

(b) Erreurs plus larges que BS/IV.

FIGURE 8 – Performance du Heston-ANN (1M filtré, Figure 9).

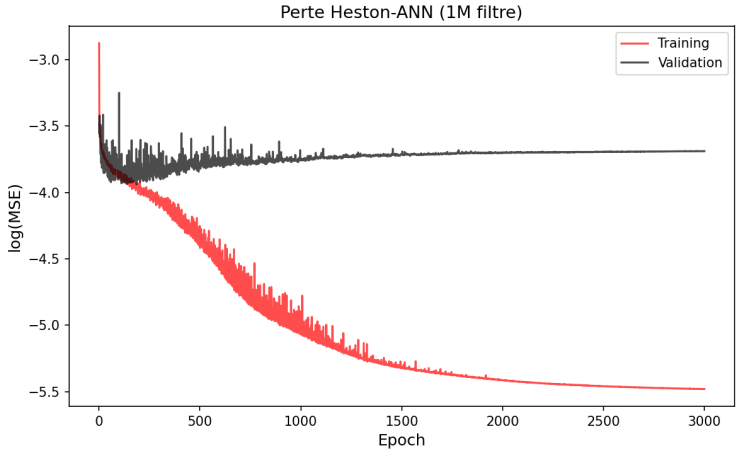


FIGURE 9 – Le surapprentissage est évident : la perte de validation (noir) diverge de la perte d'entraînement (rouge) après ∼300 époques.

4.7 Surface de volatilité implicite Heston (Tableau 11, Figures 11–12)

En pratique, les marchés cotent les options en termes de volatilité implicite plutôt qu'en prix. La *surface de volatilité implicite* — la volatilité implicite en fonction du moneyness m et de la maturité τ — est un outil central pour les traders et les gestionnaires de risque. Générer cette surface sous le modèle de Heston requiert deux étapes coûteuses : d'abord pricer l'option par la méthode COS, puis inverser la formule de Black-Scholes par la méthode de Brent. Le pipeline à deux ANN (Figure 10) remplace ces deux étapes par deux *forward passes* quasi instantanés : Heston-ANN prédit le prix, puis IV-ANN prédit la volatilité implicite.

Les Figures 11 et 12 montrent les résultats pour des paramètres Heston typiques générant un smile de volatilité ($\rho = -0{,}05$, $\kappa = 1{,}5$, $\gamma = 0{,}3$, $\bar{\nu} = 0{,}1$, $\nu_0 = 0{,}1$, $r = 0{,}02$). Deux cas sont évalués : un cas "large" (Cas 1 : $\tau \in [0{,}3; 1{,}1]$, $m \in [0{,}7; 1{,}3]$) et un cas "restreint" (Cas 2 : $\tau \in [0{,}4; 1{,}0]$, $m \in [0{,}75; 1{,}25]$). Le Cas 2, étant plus éloigné des bords du domaine d'entraînement, donne de meilleurs résultats, conformément aux observations de l'article sur les effets de bord.

La Figure 11 montre que les erreurs du pipeline sont centrées sur zéro mais présentent une asymétrie (queue gauche plus longue pour le Cas 1), réflétant l'accumulation des erreurs des deux réseaux. La surface de volatilité (Figure 12a) capture néanmoins correctement la forme du smile, et la carte de différence (Figure 12b) montre que les écarts restent inférieurs à 6×10^{-3}.

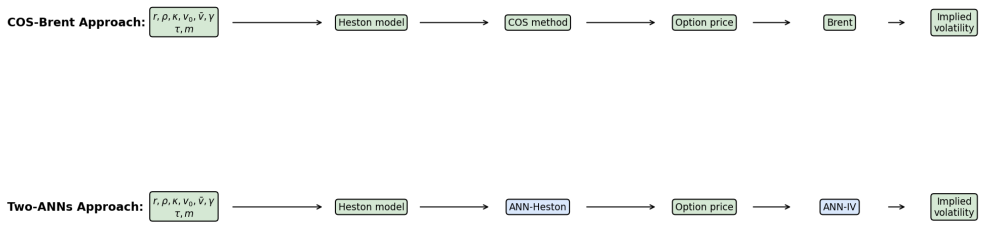


FIGURE 10 – Approche traditionnelle (COS+Brent) vs approche par deux ANN (Figure 10).

TABLE 6 – Performance du pipeline Heston-ANN + IV-ANN (Tableau 11).

	RMSE	**MAE**	**MAPE**
Notre Cas 1 ($\tau \in [0{,}3; 1{,}1], m \in [0{,}7; 1{,}3]$)	$2{,}21 \times 10^{-3}$	$1{,}67 \times 10^{-3}$	$5{,}31 \times 10^{-3}$
Notre Cas 2 ($\tau \in [0{,}4; 1{,}0], m \in [0{,}75; 1{,}25]$)	$2{,}07 \times 10^{-3}$	$1{,}62 \times 10^{-3}$	$5{,}15 \times 10^{-3}$
Article Cas 1	$7{,}12 \times 10^{-4}$	$4{,}19 \times 10^{-4}$	$1{,}46 \times 10^{-3}$
Article Cas 2	$5{,}53 \times 10^{-4}$	$3{,}89 \times 10^{-4}$	$1{,}14 \times 10^{-3}$

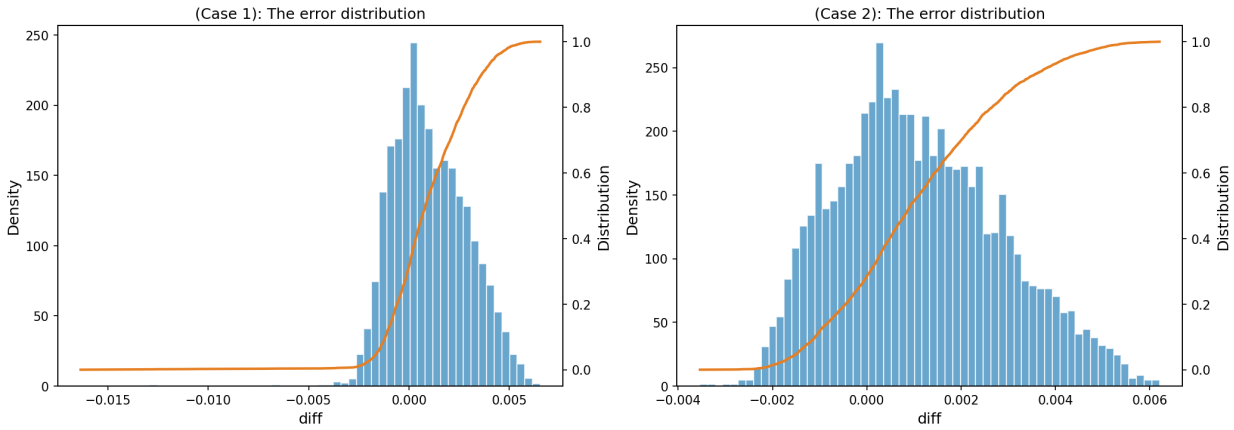
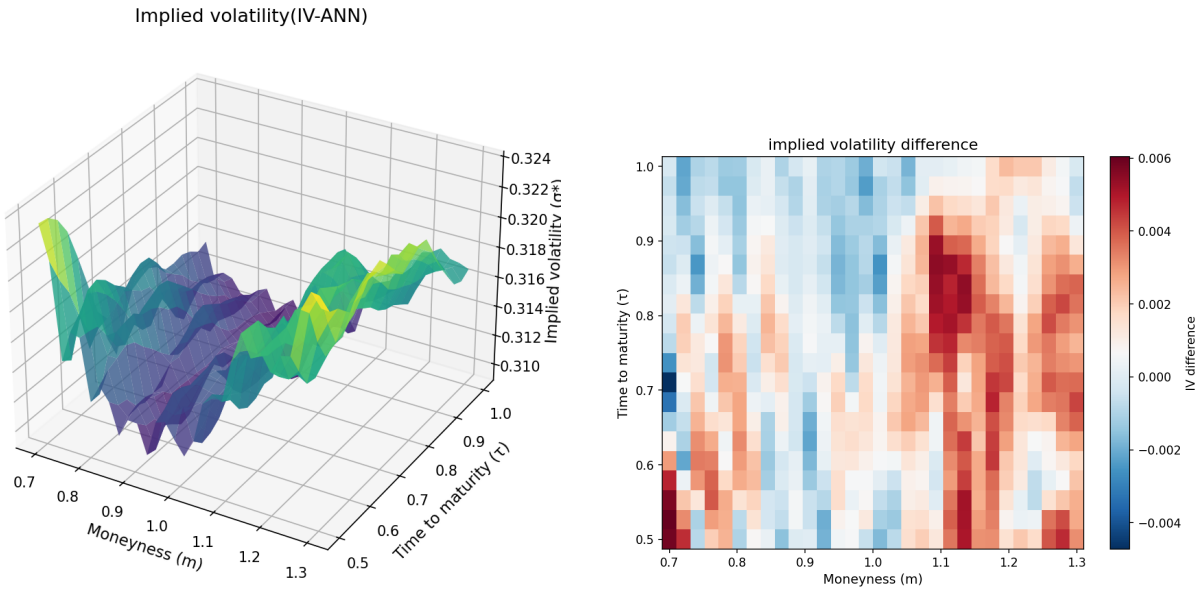


FIGURE 11 – Distributions d'erreur pour les deux cas : centrées sur zéro avec une légère asymétrie (Figure 11).



(a) Le smile de volatilité est correctement capturé.

(b) Écarts $< 6 \times 10^{-3}$ sur toute la surface.

FIGURE 12 – Surface de volatilité implicite Heston générée par le pipeline ANN (Figure 12).

5 Difficultés rencontrées et choix effectués

5.1 Pourquoi le GPU ne nous a pas aidés

L'article utilise un GPU NVIDIA Tesla P100 et rapporte des accélérations significatives. Nous avons testé deux alternatives : le GPU Apple MPS (intégré au M3) et un GPU NVIDIA T4 sur Google Colab. Dans les deux cas, l'entraînement était **plus lent qu'en CPU** : MPS prenait 46s contre 31s en CPU pour 50 époques sur 100K échantillons. Sur Google Colab avec un GPU T4, l'entraînement avec 1M d'échantillons donnait ∼10s par époque, soit 20 à 80 % plus lent que les ∼5,5–8,5s par époque obtenues en CPU selon le nombre d'entrées du modèle (5,5s pour BS/IV à 4 entrées, 8,5s pour Heston à 8 entrées) (la différence de configuration explique l'écart apparent avec le test MPS à 100K).

L'explication est liée à la granularité des opérations. Lors de l'inférence, le GPU traite 20 000 options en un seul batch — une unique multiplication matricielle massive — ce qui explique l'accélération de 200×. Lors de l'entraînement, chaque époque comprend ∼780 mini-batchs de taille 1024. Pour chaque batch, le surcoût GPU (transfert mémoire, lancement de kernel, synchronisation) dépasse le temps de calcul effectif pour un modèle aussi petit que notre MLP 4×400. Le GPU n'aurait été bénéfique qu'avec un batch beaucoup plus grand (≥ 8192) ou un modèle plus complexe, ce qui s'écarterait des spécifications de l'article.

5.2 Le défi Heston : cinq tentatives

Reproduire le $R^2 = 0{,}9999993$ de l'article pour le Heston s'est avéré être notre plus grande difficulté. Nous avons procédé par itérations successives : une première tentative avec 200K échantillons sans filtrage a donné $R^2 = 0{,}985$, avec un surapprentissage clair. Nous avons alors augmenté à 1M d'échantillons, mais le résultat a été contre-intuitivement *pire* ($R^2 = 0{,}976$). L'investigation a révélé que notre méthode COS générait des prix allant de 0 à 1,98, alors que le Tableau 9 spécifie une plage $(0, 0{,}67)$: les outliers dominaient le MSE. Une tentative de normalisation des entrées/sorties n'a pas amélioré la situation ($R^2 \approx 0{,}97$). Le filtrage à $V \in (10^{-6}, 0{,}67)$ a permis d'atteindre $R^2 = 0{,}9910$. Enfin, l'application du re-entraînement sur l'ensemble train+val (conformément à la page 9 de l'article) a marginalement amélioré le résultat à $R^2 = 0{,}9914$ (gain de +0,0004), confirmant que le plafond de performance est lié à la qualité des données plutôt qu'à la procédure d'entraînement.

Notons que le modèle Heston final utilise la planification **décroissante** (DecayLR), conformément au choix retenu par les auteurs dans l'article, bien que notre propre comparaison (Tableau 1) suggère que le LR cyclique généralise légèrement mieux. Un ré-entraînement avec le LR cyclique pourrait potentiellement réduire l'écart.

L'écart restant avec l'article est probablement dû en partie à notre implémentation de la fonction caractéristique de Heston, qui utilise la formulation originale de Heston (dite "little trap") plutôt que la version numériquement stable d'Albrecher et al., qui évite les discontinuités du logarithme complexe pour les grandes maturités ou les paramètres extrêmes. L'article confirme explicitement l'utilisation d'une "version robuste de la méthode COS" (Section 4.1), ce qui corrobore cette hypothèse. Des différences de framework (PyTorch vs probablement TensorFlow) et de matériel (CPU vs GPU P100) contribuent également à cet écart.

5.3 Récapitulatif des différences

TABLE 7 – Synthèse des différences entre notre implémentation et l'article.

Élément	**Article**	**Nous**	**Justification**
Matériel	i5 + P100 GPU	M3 CPU	GPU inutile pour ce modèle
Hyperparamètres	Random search	Tableau 2 direct	Valeurs fournies par l'article
Retrain Heston	Non explicité	Oui	Extrapolation de la procédure p. 9
Heston R^2 test	0,9999993	0,9914	COS (little trap) / hardware

Ce qui est identique : architecture (4×400 MLP, ReLU, Glorot, Adam, batch 1024), LHS, 1M échantillons (BS, IV, Heston), 3000 époques, DecayLR $10^{-3} \to 10^{-5}$, gradient-squash, COS $N = 1500/L = 50$, re-entraînement train+val, parité call-put.

6 Propositions d'amélioration

Plusieurs pistes pourraient améliorer les résultats, en particulier pour le modèle Heston. L'ajout de **connexions résiduelles** ($\mathbf{z}^{(l)} = \varphi(\mathbf{W}^{(l)}\mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}) + \mathbf{z}^{(l-1)}$) améliorerait le flux de gradient à travers les couches profondes, comme démontré par He et al. [7] dans le contexte de la vision par ordinateur. Cette technique permettrait potentiellement d'entraîner des réseaux plus profonds pour mieux capturer la complexité de la fonction de pricing Heston.

Une approche particulièrement prometteuse serait l'utilisation de **réseaux de neurones informés par la physique** (PINNs), où l'EDP de pricing est incorporée comme terme de régularisation dans la fonction de perte : $L = L_{data} + \lambda L_{EDP}$. Cette approche encoderait directement les connaissances financières dans le réseau, réduisant potentiellement les besoins en données d'entraînement — un avantage majeur pour le modèle Heston où la génération de données par la méthode COS est coûteuse.

Le **transfert d'apprentissage** entre modèles pourrait également être exploité. Puisque Black-Scholes est un cas particulier de Heston ($\gamma = 0$, $\rho = 0$, $\nu_0 = \bar{\nu} = \sigma^2$), les poids du BS-ANN pourraient servir d'initialisation pour le Heston-ANN, accélérant la convergence dans les régions de l'espace des paramètres proches de la limite Black-Scholes.

Enfin, les **Grecs** des options (Δ, Γ, Vega, etc.) peuvent être calculés directement à partir du réseau entraîné via la différentiation automatique de PyTorch (`torch.autograd`). Cela fournirait un accès immédiat aux sensibilités, utile pour la couverture de portefeuilles, sans nécessiter de différences finies numériques.

Concernant le surapprentissage observé sur le Heston, un **arrêt précoce** basé sur la perte de validation, combiné à une régularisation L2 ou du dropout, pourrait améliorer la généralisation lorsque le dataset est insuffisant par rapport à la dimensionnalité du problème. Rappelons que l'article trouve que le dropout optimal est 0 ; cependant, ce résultat a été obtenu avec 1M échantillons

sur GPU, et il est possible qu'une légère régularisation soit bénéfique dans un régime de données plus limité.

Par ailleurs, l'implémentation d'une **version robuste de la fonction caractéristique** de Heston (formulation d'Albrecher et al., utilisant $g_2 = 1/g$ pour éviter les discontinuités du logarithme complexe) pourrait améliorer significativement la qualité des données d'entraînement et, par conséquent, la précision du modèle final.

Notre expérience avec le modèle Heston suggère également deux pistes spécifiques. Premièrement, une **décomposition du domaine** pourrait améliorer la précision : au lieu d'un seul réseau couvrant tout l'espace 8D, on pourrait entraîner plusieurs réseaux spécialisés sur des sous-régions (par exemple, un pour les options ATM et un pour les deep OTM), suivant une approche de type mélange d'experts. Deuxièmement, l'**augmentation de la taille du batch** pour l'entraînement GPU mérite d'être explorée : notre analyse a montré que le batch de 1024 est trop petit pour exploiter le GPU, mais un batch de 8192 ou 16384 pourrait changer la donne en réduisant le surcoût de lancement de kernels, même si cela nécessiterait un ajustement du taux d'apprentissage (règle de mise à l'échelle linéaire).

7 Conclusion

Nous avons reproduit avec succès les principaux résultats de Liu et al. [1], en suivant fidèlement la méthodologie décrite dans l'article : même architecture (4×400 MLP), mêmes hyperparamètres (Adam, batch 1024, DecayLR), même taille de dataset (1M échantillons LHS), et mêmes paramètres de la méthode COS ($N = 1500$, $L = 50$). Pour le **BS-ANN**, notre MSE de $5{,}54 \times 10^{-9}$ est légèrement meilleur que celui de l'article ($8{,}21 \times 10^{-9}$). Pour l'**IV-ANN**, notre $R^2 = 0{,}9999986$ est très proche de la référence (0,9999998), et la transformation gradient-squash améliore le MSE d'un facteur 116 (incluant l'effet des époques supplémentaires, cf. Section 3.4). La comparaison de **vitesse** confirme l'accélération de l'ANN : 208 fois plus rapide que Newton-Raphson, avec en prime une robustesse totale sur les options deep ITM/OTM.

Le seul écart significatif concerne le **Heston-ANN** ($R^2 = 0{,}9914$ contre 0,9999993). Malgré cinq tentatives successives d'amélioration — augmentation des données de 200K à 1M, normalisation des entrées/sorties, filtrage des outliers conformément au Tableau 9, et re-entraînement sur l'ensemble train+val — nous n'avons pas réussi à combler entièrement cet écart. Nous l'attribuons principalement à notre formulation non-stable (*little trap*) de la fonction caractéristique de Heston, qui peut introduire des imprécisions numériques pour certaines combinaisons de paramètres, et secondairement à des différences de framework (PyTorch vs TensorFlow) et de matériel (CPU vs GPU P100).

Néanmoins, cette démarche itérative de diagnostic et d'amélioration constitue en elle-même un résultat pédagogique important : elle illustre concrètement les défis liés à l'apprentissage en haute dimension et la sensibilité des réseaux de neurones à la qualité des données d'entraînement.

La leçon principale de cette reproduction est que la performance dépend de manière critique de la dimensionnalité du problème : pour les modèles à 4 entrées, un ANN simple suffit à atteindre une précision quasi parfaite même sur du matériel grand public, tandis que pour les modèles à 8 entrées, la taille des données, la qualité de la génération et le matériel d'entraînement jouent un rôle déterminant.

D'un point de vue pratique, l'approche reste très prometteuse pour une utilisation en production : une fois l'ANN entraîné hors-ligne — même si cette phase prend plusieurs heures — il délivre des prix en temps réel avec un facteur d'accélération de plus de 200× et une excellente précision. Cette propriété est particulièrement adaptée aux tâches de calibration de modèles et de gestion du risque en temps réel, où des milliers de prix doivent être calculés en quelques millisecondes.

Références

[1] S. Liu, C.W. Oosterlee, S.M. Bohte. *Pricing options and computing implied volatilities using neural networks.* Risks 7(1) :16, 2019.

[2] S.L. Heston. *A closed-form solution for options with stochastic volatility.* Rev. Fin. Studies 6(2) :327–343, 1993.

[3] F. Fang, C.W. Oosterlee. *A novel pricing method for European options based on Fourier-cosine series expansions.* SIAM J. Sci. Comput. 31(2) :826–848, 2009.

[4] L.N. Smith. *Cyclical learning rates for training neural networks.* arXiv :1506.01186, 2015.

[5] R.P. Brent. *Algorithms for Minimization without Derivatives.* Prentice-Hall, 1973.

[6] M.D. McKay et al. *A comparison of three methods for selecting values of input variables.* Technometrics 21 :239, 1979.

[7] K. He et al. *Deep residual learning for image recognition.* CVPR, 2016.

[8] X. Glorot, Y. Bengio. *Understanding the difficulty of training deep feedforward neural networks.* AISTATS, 2010.

[9] D.P. Kingma, J. Ba. *Adam : A method for stochastic optimization.* ICLR, 2015.

[10] G. Cybenko. *Approximation by superpositions of a sigmoidal function.* Math. Control Signals Systems 2 :303–314, 1989.

[11] J. Bergstra, Y. Bengio. *Random search for hyper-parameter optimization.* JMLR 13 :281–305, 2012.